

Wildlife Re-Identification Research

Complete Pipeline Documentation
From Real Photos to Verified Viewpoint Labels

Dataset	GZGC — Great Zebra and Giraffe Count (Open Public Dataset)
Project	Synthetic Data Pipeline for Wildlife Re-Identification
Organization	DSAIL-ReID
HuggingFace Repo	DSAIL-ReID/synthetic-gzgc
Core Tools	SAM3, Gaussian Splatting, DeepLabCut, FiftyOne, HuggingFace
Current Stage	Human Annotation & Verification in FiftyOne
Total Notebooks	3 notebooks covering the full pipeline
Total Shards	Shards 2-9 (8 shards processed in parallel)

Pipeline at a Glance

Stage	Notebook	Input	Output
1. 3D Generation	gzgc_single_object.ipynb	Real GZGC photos + SAM3 masks	GIFs, GLB meshes, PLY gaussians
2. Keypoint Detection	run_shards.ipynb	360° GIF renders	JSON keypoint files (39 keypoints/frame)
3. Viewpoint Estimation	viewpoint_estimation.ipynb	GIFs + JSON keypoints	Reference frame CSVs
4. Human Annotation	FiftyOne Tool	Reference frames	Verified & tagged dataset

1. Project Background & Motivation

1.1 What is Wildlife Re-Identification (ReID)?

Wildlife Re-Identification (ReID) is the task of recognizing the same individual animal across multiple photographs taken at different times, locations, and viewing angles. This is critical for wildlife conservation and population monitoring — being able to automatically identify individual animals from camera trap photos removes the need for expensive and invasive physical tagging.

1.2 The GZGC Dataset

The Great Zebra and Giraffe Count (GZGC) is an open public dataset collected through a large-scale citizen science wildlife survey in Africa. It contains 4,948 real wildlife photographs of zebras and giraffes in their natural habitat, taken from various angles, distances, and lighting conditions.

1.3 The Core Challenge

Training a ReID model requires multiple labeled images of the same individual animal from different viewpoints. In the wild, this data is extremely hard to collect — you cannot control how animals are photographed. Two photos of the same zebra might look completely different simply due to viewing angle, causing a naive model to think they are different animals.

1.4 The Solution — Synthetic Data Pipeline

The solution is to generate synthetic 3D models from real wildlife photos and render them from controlled 360° viewpoints. This gives unlimited labeled training data from any angle, bridging the gap between scarce real-world labeled data and what is needed to train a robust ReID model.

Core Research Idea: Synthetic 3D renders of zebras and giraffes can be used to generate viewpoint-normalized training data sufficient for accurate wildlife re-identification.

2. Notebook 1 — 3D Model Generation

File: gzgc_single_object.ipynb

This is the first and most computationally intensive notebook in the pipeline. It takes real wildlife photographs from the GZGC dataset and converts them into 3D models, then renders 360° animated GIFs of those models. This notebook was the first one run by attachment assistants joining the project.

2.1 Pipeline Overview

Step	Description
Setup & Config	Configure shard ID, paths, HuggingFace repo, and verify dataset integrity
Load Segmentations	Load pre-computed SAM3 segmentation masks for each animal
Filter Masks	Remove masks that are too small (below 10% of image size threshold)
3D Generation	Run Gaussian Splatting inference to create 3D model from 2D photo + mask
Render GIFs	Render the 3D model rotating 360° and save as animated GIF
Upload to HuggingFace	Push all outputs (GIF, GLB, PLY) to the shared dataset repository

2.2 Key Concepts

SAM3 Segmentation

SAM3 (Segment Anything Model 3) uses text prompts like 'zebra' or 'giraffe' to automatically identify and isolate individual animals from the background in each photo. The result is a binary mask — a map showing exactly which pixels belong to the animal and which belong to the background. These masks were pre-computed and stored as RLE (Run-Length Encoding) format in a CSV file to save storage space.

Mask Size Filtering

Not all segmentation masks are useful. Animals that are very far from the camera appear small in the image and would produce poor quality 3D models. A size threshold filter removes any mask whose width or height is less than 10% of the full image dimensions.

```
Code: MASK_SIZE_THRESHOLD = 0.1 — masks below this relative size are discarded before 3D generation.
```

Gaussian Splatting (3D Reconstruction)

Gaussian Splatting is a cutting-edge technique for reconstructing a 3D model from a single 2D image. Given the photo and the animal's segmentation mask, the model generates a 3D representation using millions of tiny 3D Gaussian blobs that together form the shape and appearance of the animal. This 3D model is saved in two formats:

- PLY (.ply) — Gaussian splat point cloud format for raw 3D data
- GLB (.glb) — Standard 3D mesh format viewable in FiftyOne and 3D software

360° GIF Rendering

The 3D model is rendered from a full 360° rotation at 30fps, producing an animated GIF with 300 frames. The camera is positioned at a fixed distance with a slight upward pitch angle to simulate a realistic wildlife photography perspective. These GIFs are the primary input for all subsequent pipeline stages.

2.3 Outputs Uploaded to HuggingFace

File Type	Extension	Purpose
Animated render	.gif	360° rotation of 3D model — used for viewpoint estimation
3D mesh	.glb	Standard 3D model — used for visualization in FiftyOne
Gaussian splat	.ply	Raw 3D point cloud — stored for future use
Source images	.jpg	Original real GZGC photos — used as reference

3. Notebook 2 — Keypoint Detection (Parallel Execution)

File: run_shards.ipynb

This notebook runs DeepLabCut pose estimation on all GIF renders across multiple dataset shards simultaneously. Rather than running each shard manually one at a time, this notebook uses Papermill and Python's ThreadPoolExecutor to process all 8 shards in parallel — dramatically reducing the total processing time.

3.1 What is Papermill?

Papermill is a tool that allows you to run a Jupyter notebook programmatically as if it were a script, passing in different parameters each time. This means you can take one notebook (the keypoints detection notebook) and run it 8 times simultaneously, each time with a different SHARD_ID parameter.

3.2 Parallel Processing

The notebook uses ThreadPoolExecutor with 8 workers to run all 8 shards at the same time. Each worker processes one shard independently, running DeepLabCut inference on every GIF in that shard.

Key parameters: SHARD_IDS = [2,3,4,5,6,7,8,9] and WORKERS = 8 — meaning all 8 shards run simultaneously.

3.3 DeepLabCut Keypoint Detection

DeepLabCut is a state-of-the-art animal pose estimation library originally developed for neuroscience research. It uses a deep neural network to detect specific body landmark positions (keypoints) in each frame of a video or image sequence. For this project, it detects 39 anatomical keypoints on each animal per frame.

3.4 The 39 Keypoints

Each frame of every GIF has all 39 keypoints detected simultaneously. Every keypoint has three values: x position, y position, and confidence score.

Body Region	Keypoints
Head	nose, upper_jaw, lower_jaw, mouth_end_right, mouth_end_left
Eyes	right_eye, left_eye
Ears	right_earbase, right_earend, left_earbase, left_earend
Antlers	right_antler_base, right_antler_end, left_antler_base, left_antler_end
Neck	neck_base, neck_end, throat_base, throat_end
Back	back_base, back_end, back_middle
Tail	tail_base, tail_end
Front Legs	front_left_thai, front_left_knee, front_left_paw, front_right_thai, front_right_knee, front_right_paw

Back Legs	back_left_thai, back_left_knee, back_left_paw, back_right_thai, back_right_knee, back_right_paw
Body	belly_bottom, body_middle_right, body_middle_left

3.5 Output JSON Structure

For each GIF, DeepLabCut produces a JSON file containing keypoint detections for all 300 frames. The JSON is organized by frame, with each frame containing all 39 keypoints and their x, y, confidence values. A confidence value of -1 indicates the keypoint was not detected in that frame.

Important: A single frame contains ALL 39 keypoints detected simultaneously — like placing 39 dots on every important body part of the animal in one go.

4. Notebook 3 — Viewpoint Estimation

File: viewpoint_estimation.ipynb

This is the most algorithmically interesting notebook in the pipeline. It takes the GIF renders and their corresponding JSON keypoint files and automatically determines which frame in each GIF corresponds to which viewing direction — left, right, front, back, and the four diagonal orientations.

4.1 Why Only the Eyes?

Although 39 keypoints are available, only the left and right eye keypoints are used for viewpoint estimation. This was a deliberate and mathematically elegant decision based on three key requirements that the chosen keypoints must satisfy:

Requirement	Eyes	Legs	Nose	Ears	Back
Produces periodic signal	YES	NO	YES	YES	PARTIAL
Clean stable signal	YES	NO	PARTIAL	PARTIAL	PARTIAL
Symmetric pair	YES	NO	NO	YES	NO
Rarely occluded	YES	NO	PARTIAL	NO	YES
All requirements met	YES	NO	NO	NO	NO

4.2 The Sine Wave Insight

When an animal rotates 360° at constant speed, the x-position of each eye traces a perfect sinusoidal path. This is because a point rotating in a circle, when projected onto the x-axis, mathematically produces a sine wave. This insight — turning a computer vision problem into a signal processing problem — is the core innovation of this stage.

Key insight: The trough (minimum point) of the sine wave corresponds exactly to the frame where the animal is facing directly left — because that is when the eyes are at their leftmost x-position on the image.

4.3 Step-by-Step Algorithm

Step 1 — Load Data

For each GIF, load the corresponding DeepLabCut JSON file and reformat the keypoints from frame-organized to keypoint-organized structure for easy access.

Step 2 — Extract Eye Positions

Extract the x-coordinate of the left eye and right eye across all 300 frames, creating two arrays: lxs (left eye x-positions) and rxs (right eye x-positions).

Step 3 — Forward/Backward Fill

Some frames have missing detections (confidence = -1). These gaps are filled using forward-backward fill — replacing missing values with the nearest known value. Forward fill handles

gaps in the middle, backward fill handles gaps at the start.

Step 4 — Signal Shifting

The signal is shifted so that the trough (minimum) aligns to a standard position. This makes subsequent processing consistent across all animals.

Step 5 — Fold and Mirror

The signal is folded at the halfway point and mirrored, creating an approaching and leaving component for both eyes — giving 4 signals total for averaging.

Step 6 — Savitzky-Golay Smoothing

All 4 signals are smoothed using a Savitzky-Golay filter (window=21, polyorder=3) and averaged together. This removes noise while preserving the underlying sine wave shape.

Step 7 — Find Trough

The minimum of the smoothed averaged signal is found using `np.argmin()`. This index corresponds to the frame where the animal faces directly left — the reference frame.

Step 8 — Extract 8 Viewpoints

Starting from the trough index, 8 evenly spaced frames are extracted every 37.5 frames apart, covering the full 360° rotation with 8 canonical viewpoint directions.

4.4 The 8 Canonical Viewpoints

Viewpoint	Direction	Frame Offset from Trough
1	Left (reference)	0 frames
2	Back-Left	+37.5 frames
3	Back	+75 frames
4	Back-Right	+112.5 frames
5	Right	+150 frames
6	Front-Right	+187.5 frames
7	Front	+225 frames
8	Front-Left	+262.5 frames

5. Stage 4 — FiftyOne Visualization & Human Annotation

After the automated viewpoint estimation algorithm runs, human verification is required to catch cases where the algorithm produced incorrect results. FiftyOne is an open-source tool for visualizing, exploring, and annotating machine learning datasets.

5.1 What is Loaded into FiftyOne?

Group Name	Content	File
lxs	Left eye x-position plot over all frames	0.png
rxs	Right eye x-position plot over all frames	1.png
lrxs_ff	Both eyes after forward/backward fill	2.0.png
lrxs_ff_shifted	Both eyes after shifting	2.5.png
lrxs_fold	Folded and mirrored eye signals	3.png
smoothed_average	Final smoothed averaged signal with trough	4.png
viewpoints	Source image + 8 canonical viewpoint frames	5.png
threed	Interactive 3D GLB mesh viewer	.fo3d

5.2 The Annotation Task

Human annotators review each animal's viewpoint estimation results in FiftyOne and tag samples that look incorrect as 'bad'. A result might be bad if the trough was found at the wrong frame — for example if the eye signal was too noisy or the animal was heavily occluded in many frames.

5.3 What Happens to Bad Samples?

Samples tagged as 'bad' have their `trough_index` set to -1 in the final CSV export. This flags them as unreliable so they are not used in subsequent ReID training. The final CSV containing trough indices, dropped frame counts, and bad flags is uploaded to HuggingFace as the verified reference frames dataset.

Current Status: The project is currently at this stage — human annotation and verification of viewpoint estimation results in FiftyOne.

6. Why Viewpoint Estimation Matters for ReID Training

Viewpoint estimation is the critical bridge between the synthetic data pipeline and the ReID model training. Without it, training would be blind to one of the biggest sources of variation in wildlife photography.

Controlled Training Pairs

Knowing exactly which frame = which viewpoint allows creating labeled pairs like 'Zebra #47 from the LEFT' and 'Zebra #47 from the FRONT-RIGHT'. This structured viewpoint-specific labeling is essential for supervised ReID training.

Viewpoint-Aware Learning

The ReID model can learn that two images look different but are the same animal — just at different angles. Without viewpoint labels this distinction is impossible to teach explicitly.

Hard Negative Mining

A hard negative is a different animal at the same viewpoint. These are the most valuable training examples for metric learning because they force the model to learn fine-grained identity differences beyond just viewpoint differences.

Synthetic to Real Bridging

Verified viewpoints allow matching synthetic render frames to real wildlife photos taken from similar angles — creating cross-domain training pairs that help the model generalize from synthetic to real images.

7. Next Steps — Road Ahead

COMPLETED

- Synthetic data generation (Notebook 1)
- DeepLabCut keypoint detection on all shards (Notebook 2)
- Automated viewpoint estimation algorithm (Notebook 3)
- FiftyOne dataset setup and loading

IN PROGRESS — CURRENT STAGE

- Human annotation and verification of viewpoint results in FiftyOne
- Tagging bad/incorrect samples
- Uploading verified reference frame CSVs to HuggingFace
- Quality checking across all shards

NEXT — ReID DATASET CONSTRUCTION

- Pair synthetic renders with real GZGC images using verified viewpoints
- Create matched pairs: same individual, different viewpoints
- Build train/val/test splits — careful to avoid individual identity leakage across splits
- Construct hard negative pairs for metric learning

FUTURE — MODEL TRAINING & EVALUATION

- Train ReID model using metric learning (triplet loss, ArcFace, or contrastive learning)
- Backbone options: ResNet, Vision Transformer (ViT), or wildlife-specific models
- Handle synthetic-to-real domain adaptation challenge
- Evaluate using standard ReID metrics: mAP, Rank-1, Rank-5 accuracy
- Paper writing and results analysis

8. Tools & Technologies Reference

Tool / Library	Category	Purpose in This Project
SAM3	Segmentation	Text-prompted animal segmentation from real photos
Gaussian Splatting	3D Reconstruction	Convert 2D photo + mask into 3D model
DeepLabCut	Pose Estimation	Detect 39 body keypoints per frame in GIFs
FiftyOne	Annotation	Interactive dataset visualization and human annotation
HuggingFace Hub	Data Management	Dataset hosting, versioning, and sharing
Papermill	Automation	Run notebooks programmatically with different parameters
ThreadPoolExecutor	Parallel Computing	Run 8 shards simultaneously
SciPy savgol_filter	Signal Processing	Smooth eye position signals
NumPy	Numerical Computing	Array operations, signal processing, argmin
Matplotlib	Visualization	Plot eye movement signals and results
PIL / Pillow	Image Processing	Open and navigate GIF frames
Pandas	Data Management	Manage shard data and export CSVs
pycocotools	Mask Processing	Decode RLE format segmentation masks
PyTorch	Deep Learning	GPU-accelerated inference for 3D generation
Python / Jupyter	Development	Core development and experimentation environment

9. Key Concepts Glossary

Re-Identification (ReID)

The task of recognizing the same individual across multiple images taken at different times and angles.

Gaussian Splatting

A 3D reconstruction technique that represents a scene using millions of tiny 3D Gaussian blobs, enabling high quality 3D models from single 2D images.

Segmentation Mask

A binary image map indicating which pixels belong to the object of interest (animal) vs background.

RLE (Run-Length Encoding)

A compressed format for storing segmentation masks efficiently by encoding consecutive pixels.

Keypoint

A specific anatomical landmark on an animal's body (e.g. left eye, nose, knee) whose position is detected and tracked across frames.

Confidence Score

A value between 0 and 1 indicating how certain the model is about a keypoint detection. A value of -1 means the keypoint was not detected.

Forward/Backward Fill

A technique to replace missing values in a sequence by copying from the nearest known value forward or backward in time.

Savitzky-Golay Filter

A signal smoothing filter that fits a polynomial to a sliding window of data points, removing noise while preserving the signal's shape.

Trough

The minimum point of the sine wave signal — corresponds to the frame where the animal faces directly left.

Hard Negative

In metric learning, a hard negative is a different individual that looks very similar — e.g. same viewpoint but different zebra. These are the most valuable training examples.

Domain Adaptation

Techniques for making a model trained on synthetic data generalize to real-world data, bridging the gap between the two domains.

mAP (mean Average Precision)

A standard evaluation metric for ReID systems measuring how accurately the model ranks correct matches above incorrect ones.

Rank-1 Accuracy

The percentage of queries where the top-1 retrieved result is the correct match — the most common single metric for ReID evaluation.

Shard

A subset of the full dataset. The GZGC dataset is divided into shards (2-9) for manageable parallel processing.

Papermill

A Python tool for executing Jupyter notebooks programmatically with parameterization, enabling automated batch processing.